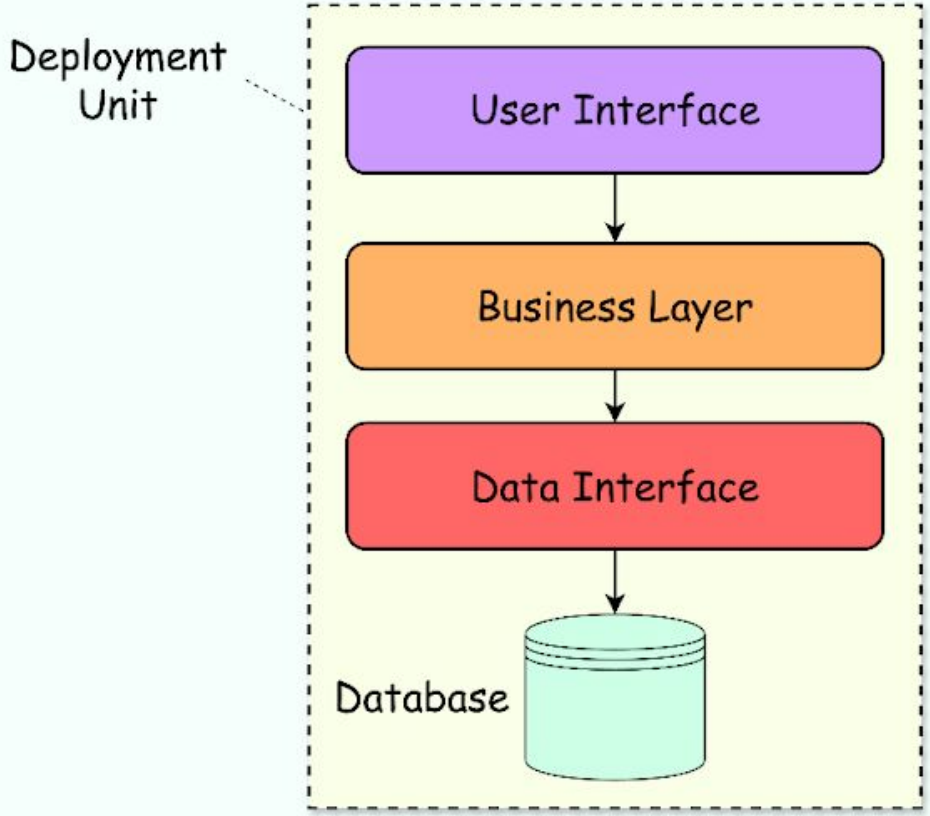
Monolith & Microservices & Load Balancing

Spring 2025

Monolith

- Single unified codebase
  - All components of the application are **tightly coupled** and **run as a single unit**
    - Authentication, business logic, UI, etc.
- Can be deployed as once.
  - Some languages even create a single binary (Go, Java)
- Components depend on each other. Scaling and updating are hard.
  - You need to update the whole codebase.
- Shared memory / database
  - Usually, the system uses a single database.
  - **Single point of failure**
- Good at start

1 Monolithic System



Monolith (cont.)

- When people think of a *traditional* monolith they often picture a Spring Boot app with Thymeleaf, Flask with Jinja2, or alike.
- A REST API can also be a monolith.
  - If it is a single, unified codebase.
  - If all domains (auth, billing, inventory, shipping) are tightly coupled in that one project.
  - If it is packaged into a single binary or artifact and deployed **all at once** to a single server or cluster.
- Example:
  - If you find a bug in *Payment* logic, and to fix it you have to *rebuild* and *redeploy* the entire backend, you are running a **monolith**.
- By default, the apps we create are **monolith**.

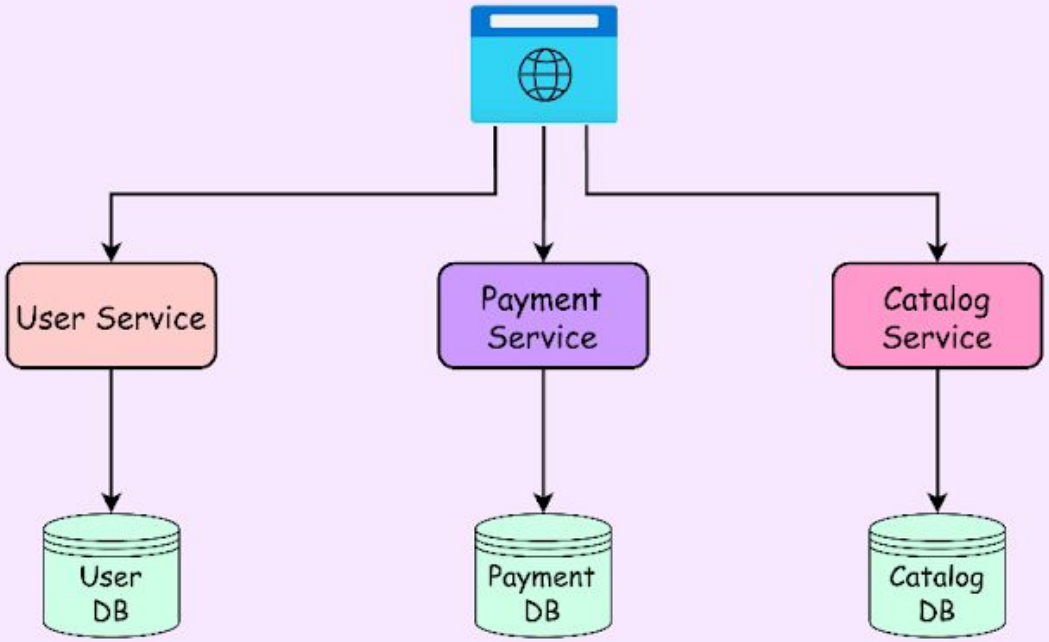
Monolith (cont.)

- You can add internal file routing and do logical separation.
  - Can have **controllers/**, **models/**, **routes/**
  - This means it is **clean and organized code**
  - Still a monolith, still all compiles (interprets) into a **single running process**.
- If one route crashes the server, the whole app goes down.
- Still deploy it as a single unit to a single server.

Microservices

- Collection of **loosely coupled**, **independently deployable** services.
  - Each service is responsible for a **specific business capability**
  - Communicates with others
    - Using HTTP APIs, gRPC or **message queues**
- Each microservice has its own codebase, business logic and/or database (optional)
  - Database can be shared, but it is an *anti-pattern* → can introduce tight coupling
- Deployment is **decoupled**
  - Every service can be deployed and scaled individually
- Different services can use **different programming languages and databases**.
- Even if there is a fault in one of the microservices, the system will carry on working
  - Provides **fault isolation**
- Scaling and replacing specific services is much **easier**.

2 Microservices Architecture



- Here, you will have different **apps**
  - Your User App will run on Server 1 (Port 5000)
  - Your Billing App will run on Server 2 (Port 5001)
- They (may) have separate databases, separate git repos.
  - They can be totally different applications.
- If they want to talk to each other;
  - They need to send requests over the network (HTTP, gRPC)

Monolith

- Simplicity
  - Single codebase, one deployment and one runtime.
- Easier development at the start
  - Simple project setup. Quicker to build MVP.
- Easier testing
  - Running end-to-end tests is much easier.
- Performance
  - Internal function calls are **faster** than **network calls** (microservices)
- Simplified Deployment
  - Single artifact. Easier to deploy
- Simple Debugging & Logging
  - Tracing logs and debugging is much easier since everything is in one place.

- Tightly coupled
  - Changing one part may break another.
  - Hard to *separate concerns*
- No selective scaling
  - You must scale the app as a whole. Cannot do it selectively.
- Slow development over time
  - As the app grows, codebase becomes hrd to manage.
  - Longer build and deploy times (when grows too much)
- Hard to adopt new technologies
  - Hard to change tech stack after time
- Reliability
  - Single point of failure
- No team autonomy
  - Everyone is working on the same codebase
  - They step on each other's toes

Microservices

- Independent deployments
  - Just deploy the microservice without redeploying the whole system.
- Scalability
  - Services can be scaled **independently**. e.g., if payment service gets more traffic, scale that only
- Resilience & Fault Isolation
  - If one service fails, others keep running.
- Tech stack
  - You are free to choose your tech stack for each microservice
- Faster dev time
  - Separate teams for microservices
  - They can own and deploy their services
- Easier to manage
  - Easier to understand since it is not a huge codebase

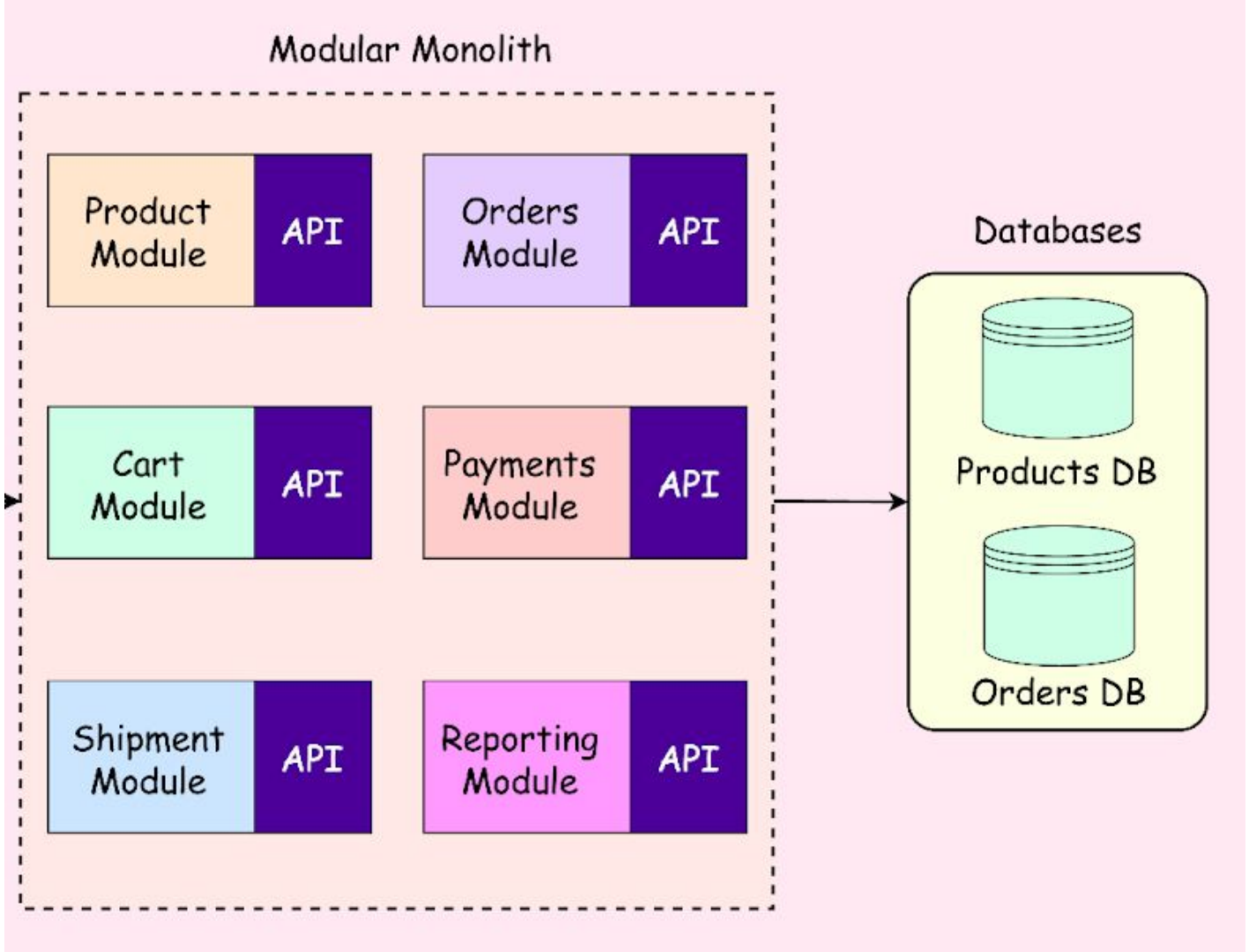
- Complexity
  - Distributed systems are hard.
  - CAP theorem
- Hard to test & debug
  - end-to-end testing is hard
  - tracking issues across services is hard
- Operational overhead
  - Needs orchestration tools (docker, kubernetes, etc)
- Data Management
  - Services often have different db's, hard for data consistency
- Deployment
  - More devops!
  - Versioning between services can be tricky
- Higher latency
  - Network calls are slower than ipc

Modular Monolith

- Divides the application logic into modules
  - Each module is **independent** and **isolated**
  - Each module have its own business logic (having a diff. db is optional)
- It is one step away from *microservices*
  - Natural order: Monolith -> Modular monolith -> Microservices
- We still **build** and **deploy** a **single** application
  - Just like traditional monolith approach
- But we divide the code into independent modules

Modular Monolith

- In a monolith, all services are **tightly coupled**.
  - No clear boundaries, separation of concerns, etc.
- In a modular monolith
  - Different services have **no direct access to each other's data**
  - Clearly defined interfaces
  - Strict boundaries
    - Each module only exposes what's necessary
- e.g. Imagine we have **payments**, **orders** and **users** services.
  - If a change in payment requires to change code in others, that means there is **tight coupling**
  - It is a monolith.


Modular Monolith
Product Module
API
Orders Module
API
Cart Module
API
Payments Module
API
Shipment Module
API
Reporting Module
API
Databases
Products DB
Orders DB

Traditional vs. Modular

- **Service** is not a separate server.
  - It is an independent module of business logic living inside the *same* application.
- To keep things clean, we divide them by **Domain Boundaries**
- Traditional Monolith
  - A global **/database** folder.
  - A global **/routes** folder.
  - The **Orders** code directly queries the **Inventory** database table.
- This is **tightly coupled**
  - If you change how *inventory* works, the *orders* code will be broken.

Traditional vs. Modular

- We divide the app into self-contained folders.
  - /inventoryModule, /orderModule
- Each module owns its own data.
- If the OrderModule needs to know if a laptop is in stock, it is **strictly forbidden** from reading the *inventory* database.
  - It must call a *public* function inside the inventoryModule.

Example

- In a traditional monolith, everything lives in one place.
- Database is shared and Order code reaches directly into Payment and Inventory tables.

Good design

- Let’s say we have several services:
  - UserService
  - OrderService
  - PaymentService
- Sometimes in code, we call all these servers from one another (service chaining).
  - OrderService calls PaymentService, it calls InventoryService, etc.
  - This is **not** a good design.
    - We should avoid deep chains of service calls.
  - This leads to **tight coupling**, **high complexity** and brittle code. A change in one ripple through the whole chain.

Good design

- So here the services rely on each other’s behavior.
- It is also hard to test.
- Hard to maintain
  - A small change may require changes everywhere.
- Single point of failure
  - If a service goes down, others will probably go down.
- Latency
  - Each call adds a latency.

Good design

- Instead of *chained service calls* we can do **orchestration**
  - We create a one service (like Main) and it coordinates everything!
- Instead of a chain, we have a main service which orchestrates.
  - This can also create a problem so be careful.
- Another solution is **event-driven**

```
OrderService {
    paymentResult = PaymentService.charge()
    if (paymentResult == success) {
        InventoryService.reserveItems()
        NotificationService.sendEmail()
    }
}
```

Good design

- Here we don’t have a central controller.
- Services **react** to events.
  - OrderService creates an OrderPlaced event.
  - PaymentService *listens* for OrderPlaced, and processes payment.
    - If successful, it creates a new PaymentCompleted event.
  - InventoryService listens to PaymentCompleted event.
    - Acts accordingly…
- This is **loosely coupled**
- It scales well.
- Easy to add new services
- But harder to debug since there is no central flow.

Summary

- First option is the chain.
  - Not a good design.
- Better one is orchestrated
  - Where we have a main service which calls and acts on others results
- Best one is event-driven
  - We'll talk about it later in detail

API Gateway

- Handles all incoming API requests from clients
  - web browsers, mobile apps, etc.
- Directs them to the appropriate backend services
  - Returns the responses to the clients
- A middleman between users and services.
- Can be used in both **microservices** and **monoliths**

API Gateway

- In **microservices**, it acts as a **Router** and **Traffic Cop**
  - Problem: Clients shouldn't have to remember IP addresses of 20 different services.
  - Solution: Client talks to *one* Gateway. Gateway looks at the URL, path and routes the request to correct backend service.
- In **monoliths**, it acts as a **Shield**.
  - **Authentication & Authorization**: Gateway checks if the user is logged in before the request ever hits the monolith server.
  - **Rate Limiting / Throttling**: Blocks spam attacks and DDOS attempts at the edge of the network.
  - **Static Caching**: Caches heavy images or files so your main server doesn't have to waste CPU processing them.

Scaling

- Scaling means **handling more load**
  - More users, more data, more traffic
- Two types of scaling:
  - Vertical Scaling (Scaling Up)
    - Adding more power to a single machine (CPU, RAM)
  - Horizontal Scaling (Scaling Out)
    - Adding more machines / nodes to spread the load.

Vertical Scaling

- We add **more power** to the existing VM.
  - However, there is a limit to adding more power!
- A single node handles the entire workload
- Relies of multi-threading on the existing machine to handle multiple requests
- Easy to apply
  - You are just running the same program on a stronger computer
- No need for **load balancing**

Vertical Scaling

- Cost-effective
  - Upgrading is cheaper than getting a new machine.
- Less complex process communication
  - Because there is a single node.
- Less complicated maintenance
  - Again, a single node
- Less need for software changes
  - You don’t change anything. Just the machine specs.
- Higher possible downtime
  - If you don’t have a backup server, in case of failure you will have downtime
- **Single point of failure**
  - Since everything is happening on a single device, on failure there will be no access to data.
- Upgrade limit
  - There is a natural limit as to improving the system.

Horizontal Scaling

- Adding **more instances / machines** to the system
- Fixes the Single Point of Failure of *vertical scaling*
- It distributes **requests** across these machines
  - How does it know which server to talk to?
    - **LOAD BALANCERS**

Horizontal Scaling

- Easier from hardware perspective
  - Instead of deep analyzing, just add a new machine
- Fewer downtime
  - If you have failure on one machine, you have others as backup
- High resilience and fault tolerance
  - Same as above.
- Increased performance
  - You can relay the user to a machine with less traffic and therefore have better speed.
- Increased complexity
  - Multiple servers are harder to maintain
  - Needs **load balancing**
  - If you want to scale the database, replication is hard.
- High initial costs
  - Adding new servers is expensive compared to improving cpu and ram

Load Balancers

- Process or technology that **distributes** traffic among several servers.
- Intercepts traffic destined for a site and redirects it to various servers.
- Performs the following:
  - Intercepts network traffic
  - Splits it into individual requests and determines which servers will receive them
  - Maintains a watch on servers, to see if they still working. If not, they are taken out of rotation.
  - Provides redundancy with failover scenarios

Load Balancers

- Three main benefits:
- **Flexibility (no downtime maintenance)**
  - Adding and removing servers is easy.
  - This allows for **maintenance**. Now we can simply apply maintenance on a machine. Imagine a *vertical scaling* scenario -> you'd have downtime.
- **High Availability**
  - Can check the status of available servers and add/remove them by itself without interaction.
- **Scalability**
  - Since it distributes the load, serving power of the system is increased.
  - By just adding new machines to the system, serving power can be increased.
    - Now, you can just have multiple small-mid level machines.

Types of Load Balancers

- **Transport Layer**
  - Works at TCP/UDP level.
  - Routes traffic based on IP address and port.
  - Fast but no insight into the **content** of the requests.
- **Application Layer**
  - Routes traffic based on the **content** (HTTP headers, URLs, cookies)
  - Smart routing
    - Can look at the URL and say: *"if the user is requesting a video, send them to video servers. If they want a webpage, send them to web servers"*

Load Balancing Algorithms

- Static Algorithms *(do not rely on the current load)*
  - **Round-Robin**: Requests are sent in a **circle**. Request 1 goes to Server A, Req 2 goes to B, ...
  - **Weighted Round-Robin:** If Server A is twice as powerful as Server B, you give Server A higher *weight* so it receives more traffic
  - **IP Hash:** Takes the IP address and mathematically map it to a specific server. Guarantees that a specific user will always hit the same server! (Very good for **sticky sessions**)
- Dynamic Algorithms (*react to real time load*)
  - **Least Connection:** Finds the server with least active users and sends the request there.
  - **Least Response Time:** Sends the req. to whichever server is answering requests the fastest.

Sticky sessions

- Before, we said that sessions are usually stored **in memory**.
  - I logged in using **Server 1**.
  - Later, when I requested something my request went to **Server 2**.
  - But it doesn't have my session → because it was in **Server 1**.
- For that reason, we use **sticky sessions**
  - We may also choose to store sessions in the database.
  - Or if we have multiple database, we may store the session in a **centralized** store such as **redis**.
- That is why IP hash is good for *sticky sessions*
  - Because since my IP is going to be the same, I will be routed to the same server!

Static Algorithms

- These algorithms **do not** rely on the **current load** of the servers.
- Round robin
  - Having **n** servers, first req goes to S1, second goes to S2, .. so on.
- Weighted round robin
  - Weights are given to servers, so that if we have a stronger server, it may receive more traffic.
- IP Hash
  - Makes sure the same IP is routed to the same server.
  - Good for **sticky sessions**.
- URL / Path Hash
  - Often used by CDN's and caching to ensure that same resource is always routed to the same server.
- Consistent Hashing
  - Will see in detail.

Dynamic algorithms

- These algorithms **react** to the real time load of servers.
- Least connections
  - The request is routed to the server with **fewest active connections**
  - Good for long-lived connections (web sockets, db queries)
- Weighted least connections
  - Adds weights to servers based on their capacity.
  - Routes the traffic on both **least conn** and **server power**
- Least response time
  - Routes traffic to server with **lowest average server resp time**
- Resource based
  - Custom, checks CPU, memory etc.
  - Sends to those with more free resources

CDN: Content Delivery Network

- Network of geographically distributed servers
  - Deliver static content (sometimes dynamic)
    - images, videos, css, js files, web fonts, etc.
  - Based on geo location, server health and load
- Purpose is to:
  - Reduce latency
  - Reduce load on the origin server
  - Improve speed, availability, and scalability
- CDN providers have **edge servers** in many geolocations.
  - They have cache copies of static content
- When the user makes a request, instead of going to origin server, user gets the response from the nearest edge server.
  - If the edge does not have in, origin server returns it.

Consistent Hashing

- Next week

Scaling in monoliths

- Typically **vertical scaling**
  - You just deploy the same app on a better and stronger machine.
- Easy to implement.
  - No need for complex orchestration or **distributed systems**
- However, there are limits to vertical scaling.
  - If some component is having problems, you cannot scale it separately.
- Vertical scaling can waste resources
  - Because we are also scaling some parts that we don't even need.

Scaling in monoliths

- We can also do **horizontal scaling**
- We deploy the **entire app** on multiple instances.
- We use a **load balancer** to distribute incoming traffic.
- All instances **share the same database**
  - This can be a bottleneck!
- Remember sessions?
  - We used to store them on RAM.
  - So now, if we are storing sessions **in-memory**, what will happen when a request is sent to another server?
  - We must have **sticky sessions** or **external session store**

Scaling in microservices

- Can use both vertical and horizontal
- Horizontal is more preferable
  - Just add more instances for the same service and apply load balancing
  - Now if you have a specific service which is under load, we can just scale **that** one
- Most microservices deployments do auto-scaling based on:
  - CPU usage
  - Memory usage
  - Custom metrics (number of reqs.)
- Stateless services are easier to scale.
  - If you need state use Redis (1) or sticky sessions (2).
- Since services call each other, you may need to protect against cascading failures
  - One calling another calling another can cause problems if one fails
  - Circuit breakers, rate limiters, etc. (will see more later)